

Multiple GPUs Parallel 4-opt 5-opt 6-opt with Multiple Variable λ -opt moves for Traveling Salesman Problem

Wen-Bao Qiao^{a,*}, Jean-Charles Créput^b, Nan Wang^a, Kun Meng^a

^aComputer School, Beijing Information Science and Technology University, China

^bCIAD, Univ. Bourgogne Franche-Comté, UTBM, F-90010 Belfort, France

Abstract

The 2-opt, 3-opt, 4-opt, 5-opt, 6-opt heuristics are classical k -opt local search algorithms for traveling salesman problems (TSP) in combination optimization area. However, these algorithms exist two drawbacks, firstly their serial implementations have polynomial time complexity and are time-consuming to approach large scale TSP instances; secondly k must be specified in advance. To solve the first drawback, some researchers turn to GPU and find different GPU parallel 2-opt and 3-opt algorithms. As to the second drawback, some researchers proposed *variable* λ -opt algorithms. Problem is that we do not find previous research on computation acceleration of 4-opt 5-opt 6-opt and variable λ -opt using GPU cards. Here in this paper, we propose multiple GPUs' parallel 4-opt 5-opt 6-opt and variable λ -opt ($\lambda = 2, 3, 4, 5, 6$) algorithms. Their parallel computation procedure follows characteristic of "data parallel" and can be easily deployed on any quantity of GPUs. Furthermore, we propose a serial algorithm that executes massive variable λ -opt moves on the same TSP tour in case of searching 2-/3-/4-/5-/6-opt simultaneously. We test these pure k -opt algorithms on national TSP instances without considering any nearest neighborhood information. Experimental tests on the mu1979.tsp instance show that when only comparing running time of one iteration of 4-opt on a 3-optimal TSP tour, the GPU parallel 4-opt runs more than 2880 times faster than serial pure 4-opt algorithm (53s vs >48h). Leading to the results that when the goal is to achieve 4-optimal TSP tour, iterative GPU parallel fixed 4-opt method runs more than 24 times faster than iterative serial fixed 4-opt (2.405h vs >56.742h), iterative GPU parallel increasing variable λ -optimal ($\lambda = 2, 3, 4$) method runs more than 58 times faster than its serial version (0.8312h vs >48h). And given enough quantity of GPU cards, the proposed GPU parallel 5-opt and 6-opt

*Corresponding author

algorithms can achieve 5-/6-optimal TSP tour in acceptable time.

Keywords: Parallel 4-opt; Massive 4-opt moves; Parallel 5-opt; Massive 5-opt moves; Parallel 6-opt; Massive 6-opt moves; Variable λ -opt; Massive variable k-opt moves; GPU.

1. Introduction

The traveling salesman problem (TSP) [1][2][3] is one of the combinatorial optimization problems that have been classified as NP-hard [4]. One solution to a TSP instance is one permutation of input n cities, while different possible permutations form various solutions. According to the permutation
5 order, the salesman travels each city once and returns to its starting city. Symmetric TSP forms un-directed graph where distance between every two cities is the same.

To optimize a given initial symmetric TSP tour, one of the most successful methods for generating optimal or near-optimal TSP solutions, namely the Lin-Kernighan heuristic (LKH) algorithm[5], has an operator called **variable λ -opt** where in each step λ edges of the current tour are replaced
10 by λ edges in such a way that a shorter tour is achieved. One single valid λ -opt exchange includes following two sub-procedures:

- Search one λ -opt, make sure the new tour is unbroken and shorter than the old one.
- Execute the selected λ -opt, this execution may reverse permutation order of the old tour.

Obviously, only executing one λ -opt exchange can not generate optimal or near-optimal TSP
15 tour, LKH adopts iterative λ -opt strategy in order to achieve a status called **λ -optimal**. A tour is said to be λ -optimal if it is impossible to obtain a shorter tour by replacing any λ of its edges by any other set of λ edges [5].

Therefore, given an initial TSP tour and in order to compute its λ -optimal status, using pure λ -opt algorithm has to search and execute λ -opt iteratively on the global TSP tour until it can not
20 find any λ -opt that can make the tour shorter. This procedure includes following two stages:

1. Search and execute one λ -opt on the whole TSP tour: check **all possible λ edges** to find one λ -opt that can make current tour shorter and produce a new integral tour.
2. Iteratively repeating step 1: repeatedly checking all possible λ edges on the global tour, until the algorithm **can not find any one λ -opt**.

25 Problem is that quantity of all possible λ edges increases rapidly as the number of cities increases, and the above step 1 takes serial $O(n^\lambda)$ time complexity to search one λ -opt. Furthermore, as to the above step 2, there is no non-trivial upper bound of the number of λ -opt need to be executed until achieving the status of λ -optimal [5].

When facing this problem, LKH considers nearest neighbor information between each two cities
30 and adopts strategies such as Euclidean minimum spanning tree(EMST [6]) and tour segment partitioning methods that reduce λ -opt searching range to decrease the algorithm's running time. Here in this paper, we search λ -opt on the global TSP tour and use GPUs to decrease running time. To achieve this goal, our contribution in this paper includes following aspects:

1. GPU parallel variable or fixed 4-/5-/6-opt algorithm with all possible valid pure 4-/5-/6-opt
35 re-connection schemes.
2. A solution that can executes multiple variable or fixed λ -opt moves on the same TSP tour.
3. Deploy the above two operations on computers with any quantity of GPUs.

Though it exists various parallel 2-/3-opt strategies in literature, we do not find an existing solution to GPU parallel 4-opt 5-opt 6-opt or variable λ -opt or solution that executes multiple
40 variable λ -opt ($\lambda=2/3/4/5/6$) moves on the same TSP tour.

The following paper is organized as this: Section 2 introduces preliminaries such as definition of fixed k-opt and variable λ -opt; Section 3 presents related parallel k-opt algorithms in literature; Section 4 introduces our proposed "multiple GPU parallel 4-opt 5-opt 6-opt local search with multiple variable λ -opt moves". Section 5 shows experimental results and comparisons. Section 6
45 concludes this paper.

2. Preliminaries

Given a symmetric Euclidean TSP instance, let V be a set of n cities and $d(u, v) \in \mathbb{R}$ a distance for each pair of cities $u, v \in V$. Let S be a set of all possible permutation solutions of a complete un-directed graph $(V, V \times V)$ and $s \in S$ a solution. Let $\pi : \{0, \dots, j, j+1, \dots, N-1\} \rightarrow V$ be a
50 bijection. A TSP tour solution $s = (\pi_0, \pi_1, \pi_2, \dots, \pi_j, \pi_{(j+1)}, \dots, \pi_{(n-1)})$ indicates that the input n cities have been permuted with an increasing permutation order from 0 to $(n-1)$ along current tour direction. Edge e_j indicates that the salesman travels from city π_j to π_{j+1} in order to avoid

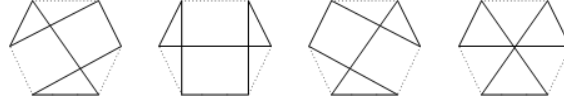


Figure 1: The 4 pure 3-opt re-connection schemes.

traveling one city twice, $(j+1) \bmod N = 0$ in case of $j = (n-1)$. The goal is to find a permutation solution $s \in S$ that minimize the cost function $f(s) = \sum_{k=0}^{n-1} d(e_j)$.

55 2.1. Fixed k -opt

Consider two neighbor TSP tour solutions $s_i, s_{i+1} \in S$, if s_{i+1} would be shorter than s_i by removing k edges and inserting k edges, this transition is defined as a **k -opt** $\mathcal{N}_k^{opq\dots}$. Searching one k -opt move should consider related cities' tour order. For example, given two oriented edges (e_o, e_p) and four related cities $o = \pi_o, o+1 = \pi_{o+1}, p = \pi_p, p+1 = \pi_{p+1}$, the 2-opt algorithm should
60 check whether $d(o, o+1) + d(p, p+1) > d(o, p) + d(o+1, p+1)$ to remain the tour integral, which is called one **valid 2-opt check** or **valid 2-opt evaluation**.

3-opt $\mathcal{N}_3$, 4-opt $\mathcal{N}_4$, 5-opt $\mathcal{N}_5$, 6-opt $\mathcal{N}_6$ or fixed k -opt $\mathcal{N}_k$ works in similar fashion with 2-opt. One small problem is that removing k edges from the old tour produces k sub-tours and leads to potentially $2^{k-1} \times (k-1)!$ re-connection schemes for k -opt, but for many of these the re-connection
65 sets are degenerate [7]. For example, as illustrated in Fig.1, only 4 of 8 re-connection schemes are pure 3-opt, others are either 2-opt or not valid. Only 25 of 48 re-connection schemes are pure 4-opt as illustrated in Fig.2 [8]. However, about 5-opt and 6-opt, we do not find in literature that detailed explain their valid re-connection schemes.

After executing the selected k -opt re-connection scheme, one or more sub-tours' original permutation order has been reversed. This leads to a necessary operation to re-order cities' permutation
70 order.

2.2. Variable λ -opt

Variable λ -opt is described as this in [5]: at each iteration step the algorithm examines, for ascending values of λ , whether an interchange of λ links may result in a shorter tour. At each
75 step the algorithm considers a growing set of potential exchanges (starting with $\lambda = 2$). These exchanges are chosen in such a way that a feasible tour may be formed at any stage of the process.

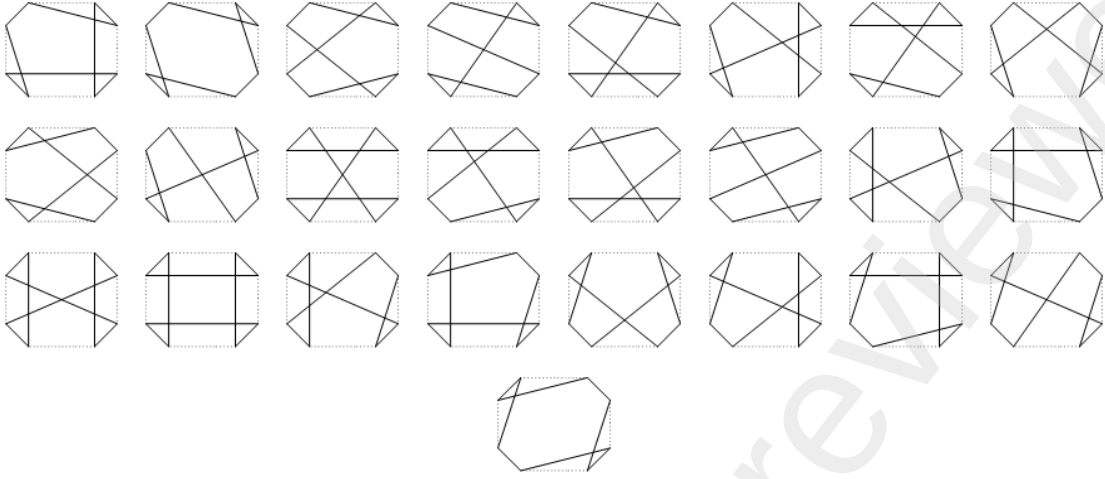


Figure 2: The 25 pure 4-opt re-connection schemes.

If the exploration succeeds in finding a new shorter tour, then the actual tour is replaced with the new tour [5].

3. Related parallel k -opt algorithm

To reduce computation time of pure k -opt algorithm, researchers have proposed various parallel 2-opt or 3-opt solutions [9, 10, 11, 12, 13, 14, 15, 16, 17]. Here, we firstly classify these related parallel k -opt algorithms, then briefly introduce our previously proposed “multiple k -opt evaluation, multiple k -opt moves” method.

3.1. Categories of parallel k -opt algorithm

Existing parallel 2-/3-opt algorithms can be classified into following three categories:

1. Search and execute multiple 2-/3-opt moves in parallel on disjoint partitions of one TSP tour [10] or in disjoint partitions of the Euclidean space [9, 13];
2. Search 2-/3-opt in parallel on the global TSP tour, but only execute the best one 2-/3-opt move[12];
3. Search and execute multiple 2-/3-opt moves in parallel on the global TSP tour or in the integral Euclidean space, named “multiple k -opt evaluation, multiple k -opt moves” [14, 16, 17].

However, none of these existing work studies parallel 4-opt 5-opt 6-opt or variable λ -opt to get λ -optimal tour when $\lambda \geq 4$. Our newly proposed methods in this paper together with our previous work “multiple k -opt evaluation, multiple k -opt moves” [16, 17] in the year 2018 can be classified into a meta-heuristic called DVND. Rios et al.[14] propose this distributed variable neighborhood descent (DVND) meta-heuristic on multi-GPU system, which includes a neighborhood search strategy called Multi-Improvement. The core ideas of DVND is to take advantage of multi-GPU massive parallelism and get multi-improvement of local search procedure such as 2-opt. While, DVND is a meta-heuristica algorithm, for example, it verifies conflict between a candidate move and the remaining moves[14], so that each verification goes over all the remaining moves and takes quadratic time complexity to judge conflict between multiple moves. Our proposed GPU parallel methods dive into much more details such as how to search massive λ -opt on the global TSP tour and how to judge non-conflict variable λ -opt moves using linear time complexity.

3.2. Brief introduction of our previous work

Here we briefly introduce our previous work “multiple k -opt evaluation, multiple k -opt moves” in [16, 17], which extends from Rock’s GPU parallel 2-/3-opt in [11, 12].

3.2.1. Rocki’s GPU parallel 2-/3-opt

The core of Rocki’s GPU parallel 2-/3-opt algorithm is the method to distribute all possible 2 or 3 edges of current global TSP tour, namely total $\frac{n*(n-1)}{2}$ 2-opt and $\frac{n*(n-1)*(n-2)}{6}$ 3-opt checks, to $O(n)$ CUDA threads in one iteration [12]. For example, the core mechanism for 2-opt is shown in Fig.3, which indicates that the total $\frac{n*(n-1)}{2}$ 2-opt checks are distributed in the boxes of Fig.3. Every pair of numbers in a box of Fig.3 indicate current tour order of two cities π_o and π_p separately. Due to the algorithm works on data structure like an ordered array of coordinates, each number in the box actually represents one oriented edge along current tour, so that each 2-opt check can be oriented same with current tour orientation.

While, the drawback of Rocki’s algorithm appears in the iterative 2-/3-opt procedure, it only executes the best one 2-/3-opt move after one iteration’s total $\frac{N*(N-1)}{2}$ 2-opt or $\frac{N*(N-1)*(N-2)}{6}$ 3-opt checks. This drawback is obvious when the goal is to obtain λ -optimal TSP tour.

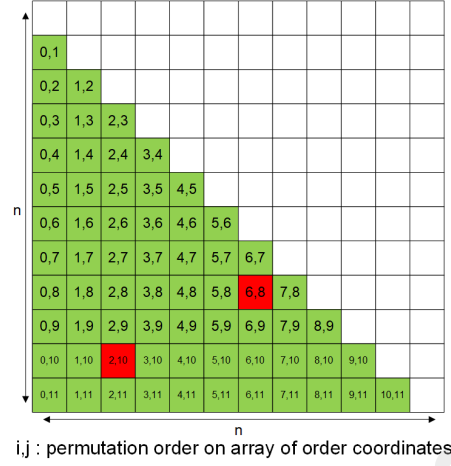


Figure 3: GPU parallel 2-opt proposed by Rocki [12] to perform complete $\frac{n*(n-1)}{2}$ 2-opt checks in parallel for a TSP tour with n input cities.[17]

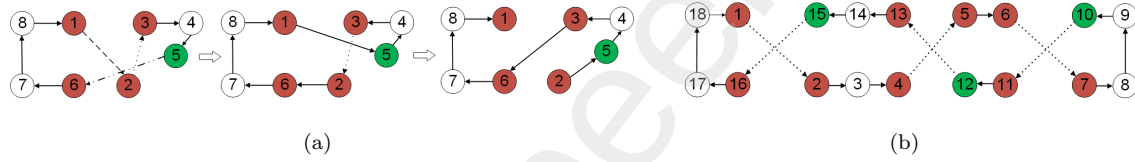


Figure 4: Cases of multiple 2-opt moves found on the same TSP tour. (a) One case that multiple 2-opt moves interact with each other: 2-opt $\mathcal{N}_2^{1,5} \rightarrow (e_1, e_5)$ interacts with $\mathcal{N}_2^{2,5} \rightarrow (e_2, e_5)$. (b) One case that multiple 2-opt moves do not interact with each other.[17] ① represents one city's tour order.

3.2.2. Our previous “multiple k -opt evaluation, multiple k -opt moves”

120 In our previous “multiple k -opt evaluation, multiple k -opt moves” method[16, 17], the k is fixed to be 2 and theoretically extends to be 3. We took advantage of Rocki’s GPU parallel 2-opt algorithm and proposed a method that takes linear time complexity to judge non-interacted 2-opt moves on the same global TSP tour. It achieves 2-optimal status of an initial TSP tour quickly since it executes massive non-interacted 2-opt moves after one iteration’s total $\frac{n*(n-1)}{2}$ 2-opt checks.

125 Theory behind that “multiple k -opt evaluation, multiple k -opt moves” includes two aspects. Firstly, one iteration of Rocki’s GPU parallel 2-opt algorithm actually finds massive independent valid 2-opt moves on the global TSP tour. Secondly, as shown in Fig.4, executing two 2-opt moves on the same TSP tour may break that tour (Fig.4(a)). So we proposed algorithm to select these non-interacted 2-opt moves (shown in Fig.4(b)) that can be executed simultaneously.

130 4. Parallel 4-opt 5-opt 6-opt with massive variable λ -opt moves

Given an initial TSP tour and previous GPU parallel 2-opt 3-opt solutions, in order to realize multiple GPUs parallel variable λ -opt with multiple variable λ -opt ($2 \leq \lambda \leq 6$) moves on the global TSP tour, we have to solve following four problems:

- [1] All possible valid re-connection schemes of pure 5-opt and 6-opt;
- 135 [2] GPU parallel algorithms to search variable or fixed 4-opt 5-opt 6-opt on the global tour;
- [3] Linear time complexity method to find non-interacted variable or fixed λ -opt ($2 \leq \lambda \leq 6$) on the same tour;
- [4] Method to deploy the above algorithms on any quantity of GPU cards.

4.1. All possible re-connection schemes of 5-opt and 6-opt

140 After removing k edge from an old TSP tour, the algorithm has to re-connect k sub-tours by k new edges and ensure the new tour is valid. Except for the pure 2-/3-/4-opt re-connection schemes that have been studied in literature, we have to find all pure 5-/6-opt re-connection schemes. The problem is that it exists $2^{k-1} \times (k-1)!$ re-connection schemes for k -opt and not all of them are valid, which is impossible to enumerate all pure 5-/6-opt re-connection schemes by hands.

145 We transfer this problem into number games. As shown in Fig.5, it exists 10 nodes need to be re-connected for 5-opt and 12 nodes need to be re-connected for 6-opt. Then the problem changes to be all possible connection schemes starting from node 1 ending with 10 or 12 and ensuring that all nodes are connected using one integral tour.

We use pseudocode shown in Alg.1 to finally find 208 pure valid 5-opt re-connection schemes as shown in Fig.6. When setting $E = 12$ in Alg.1 and adapt Alg.1 to 6 edges, we finally find 1935 valid pure 6-opt re-connection schemes as shown in Fig.7 for one set of 6 edges ($e_o, e_p, e_q, e_j, e_w, e_t$).

4.2. GPU parallel 4-opt 5-opt 6-opt and variable λ -opt

Here we firstly propose GPU parallel pure 4-opt 5-opt 6-opt solutions and then merge them into GPU parallel variable λ -opt where $2 \leq \lambda \leq 6$.

Algorithm 1 The method that we compute all possible valid re-connection schemes of 5-opt.

Require: Number $E = 10$ for 5-opt $\mathcal{N}_5^{opqjw}$.

```

1:  $o \leftarrow 1$ ;
2: for  $o' = 3; o' < E + 1; o' ++$  do
3:   EDGE  $e_1 \leftarrow (o, o')$ ; // first new edge  $e_1$ ;
4:   for  $p = 2; p < E + 1; p ++$  do
5:     if  $p! = o \& \& p! = o'$  then
6:       for  $p' = 2; p' < E + 1; p' ++$  do
7:         if  $p'! = o \& \& p' = o' \& \& p'! = p$  then
8:           EDGE  $e_2 \leftarrow (p, p')$  // second new edge  $e_2$ ;
9:           for  $q = 2; q < E + 1; q ++$  do
10:            if  $q! = o \& \& q! = o' \& \& q! = p \& \& q! = p$  then
11:              for  $q' = 2; q' < E + 1; q' ++$  do
12:                if  $q'! = o \& \& q' = o' \& \& q'! = p \& \& q'! = q$  then
13:                  EDGE  $e_3 \leftarrow (q, q')$  // second new edge  $e_3$ ;
14:                  ..... // similar way to loop from 2 to E+1 to get edges  $e_4$   $e_5$ ;
15:                end if
16:              end for
17:            end if
18:          end for
19:        end if
20:      end for
21:    end if
22:  end for
23: end for
24: Delete combinations of 5 edges  $(e_1, e_2, e_3, e_4, e_5)$  that obviously cut the original TSP tour.
25: for each combination of new 5 edges  $(e_1, e_2, e_3, e_4, e_5)$  do
26:   valid  $\leftarrow$  Start from number 1 of edge  $e_1$ , traverse all other four edges  $e_2, e_3, e_4, e_5$ , and end
   with number 10;
27:   if valide then
28:     This is a valid 5-opt re-connection scheme;
29:   end if
30: end for

```

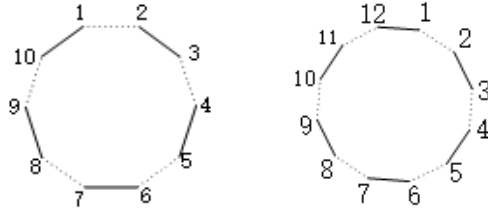


Figure 5: Removing 5 or 6 edges from the old tour, producing 10 or 12 nodes need to be re-connected.

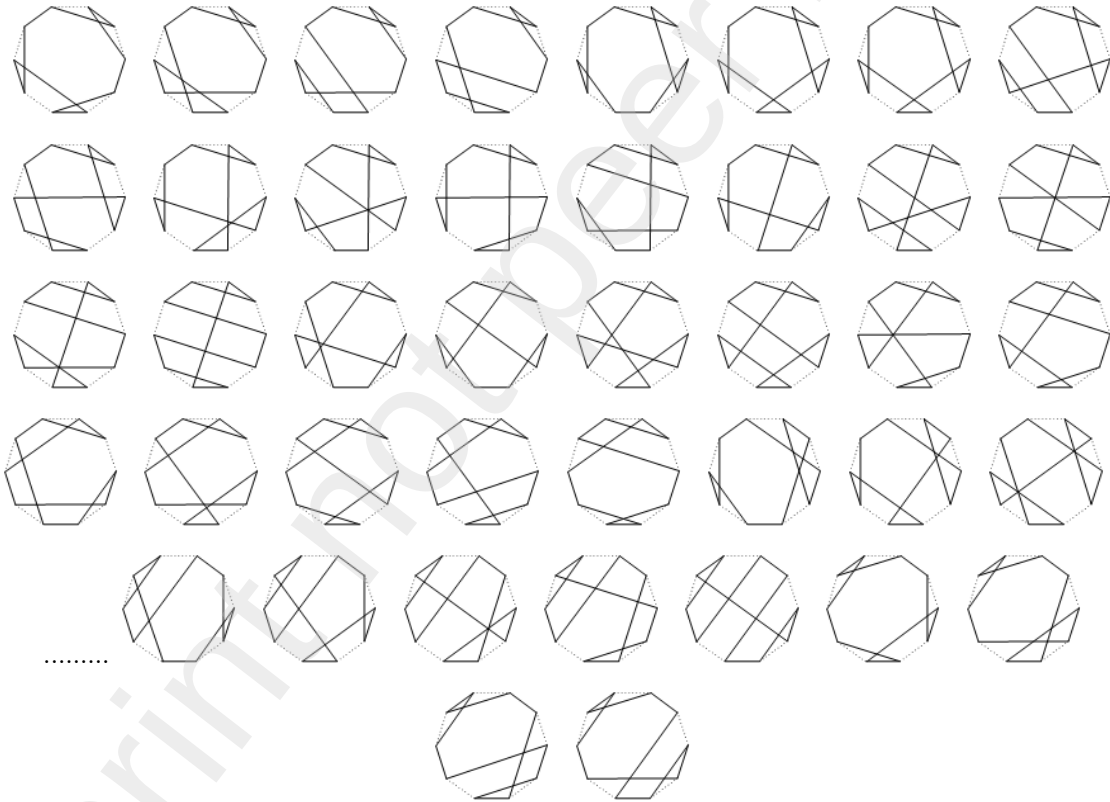


Figure 6: The 39 of total 208 pure 5-opt re-connection schemes.

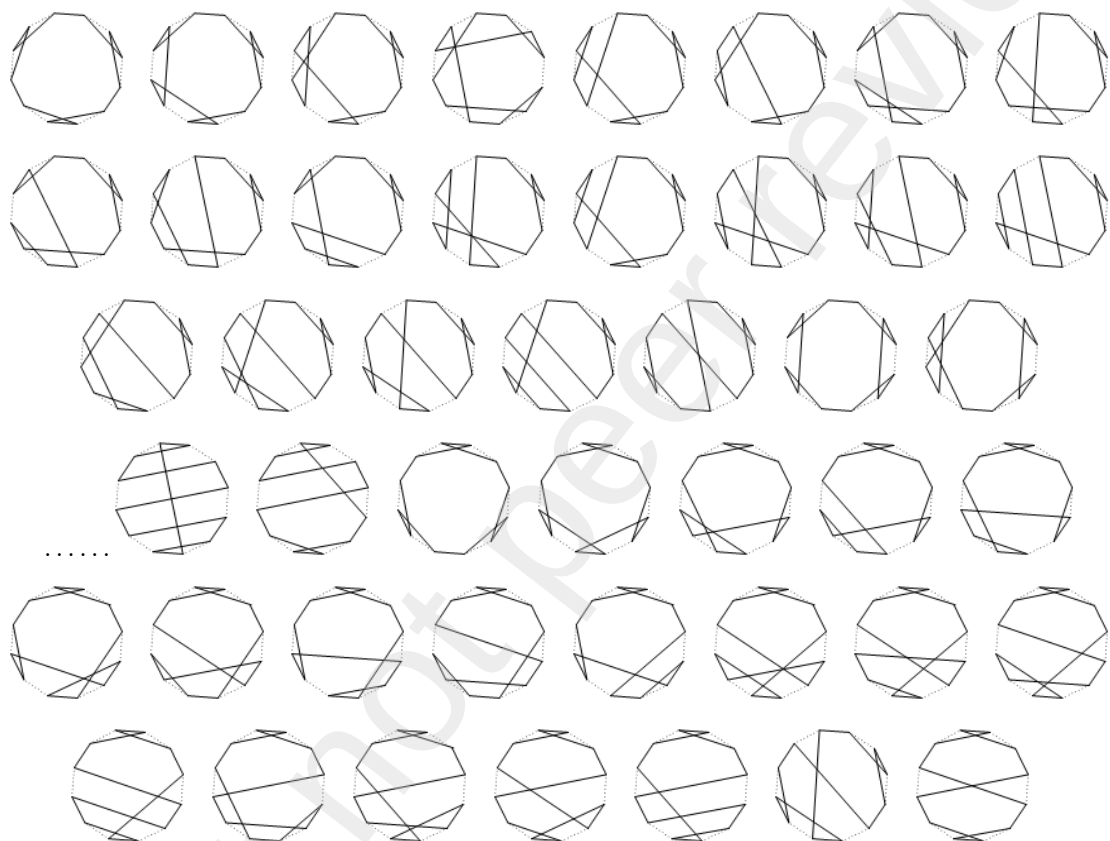


Figure 7: The 43 of total 1935 pure 6-opt re-connection schemes.

155 4.2.1. GPU parallel 4-opt and 5-opt

Given a TSP tour with n nodes, there are totally $n(n-1)(n-2)(n-3)/24$ possible 4 edges that should be checked along the global tour for 4-opt, and totally $n(n-1)(n-2)(n-3)(n-4)/120$ possible 5 edges that should be checked for 5-opt. Our GPU parallel 4-/5-opt solution aims to deploy all these 4-/5-opt checks on GPU side.

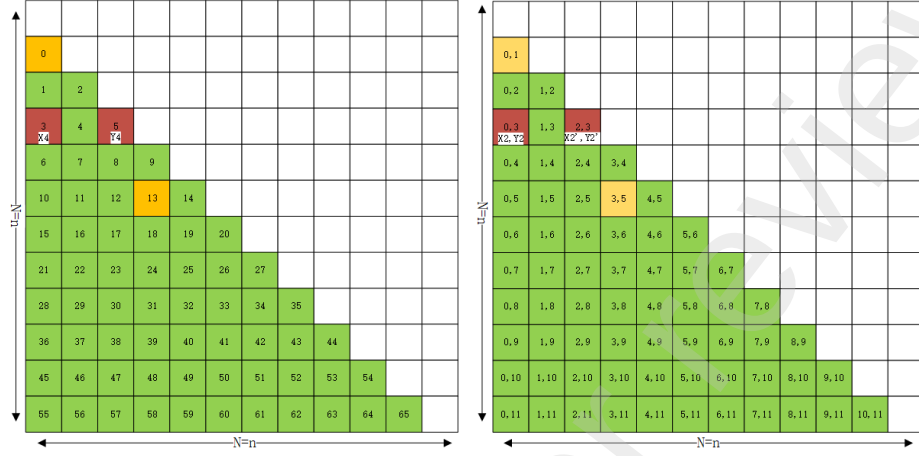
160 We found that it is difficult to find a solution to a quartic equation with a coefficient of one dollar for 4-opt and also difficult to solve a quintic equation with a coefficient of one dollar for 5-opt. Instead of solving these quartic or quintic equation, we find a GPU parallel 4-opt and 5-opt solution that builds connections between all possible 4-/5-opt edges and CUDA threads.

Our proposed GPU parallel 4-/5-opt algorithm extends from Rocki's GPU parallel 2-opt. Its 165 procedure is shown in Fig.8. Supposing current TSP tour is oriented with n nodes and n edges, all possible $n(n+1)/2$ pairs of edges are distributed in Fig.8 (a,b), they can be mapped into CUDA threads easily same as Rocki's method.

For 4-opt $\mathcal{N}_4^{opqj}$, we let $N = n(n+1)/2$ and map all $N(N+1)/2$ numbers into CUDA threads as shown in Fig.8(c), each box in Fig.8(c) represents one number $0 \leq id_{(X4,Y4)} \leq N(N+1)/2$, 170 each $id_{(X4,Y4)}$ can be splited into two numbers $0 \leq X4, Y4 \leq n(n+1)/2$, a single $X4$ or $Y4$ can be parsed into two numbers in Fig.8(a), which finally represent four edges along current TSP tour as shown in Fig.8(b). Once one CUDA thread gets 4 edges of current TSP tour, this CUDA thread searches all 25 pure 4-opt re-connection schemes shown in Fig.2, and accomplishes one 4-opt check.

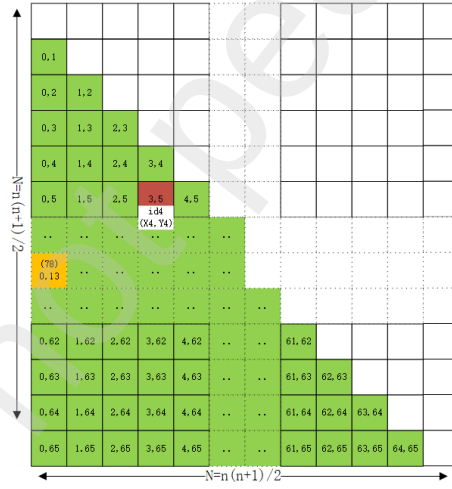
For 5-opt $\mathcal{N}_5^{opqjw}$, if we continue to extend the search range, for example let $N' = N(N+1)/2$ 175 and map all $N'(N'+1)/2$ checks into CUDA threads, this number will be too large and can produce 8 edges for 8-opt. Instead, we adopt a simple method that just loops $(n-j)$ times of each valid 4 edges in the CUDA kernel, as shown in Alg.2. Once one CUDA thread gets 5 edges of current TSP tour, this CUDA thread searches all 208 pure 5-opt re-connection schemes shown in Fig.6 and accomplishes one 5-opt check.

180 Here, we just need to prove the above proposed GPU parallel 4-/5-opt method can cover all possible $n(n-1)(n-2)(n-3)/24$ four edges and $n(n-1)(n-2)(n-3)(n-4)/120$ five edges along the global TSP tour. We take an example shown in Fig.8(c), the two numbers in each box of the first column in Fig.8(c) represent four edges, the first number 0 represents two edges $e_0(\pi_0, \pi_1), e_1(\pi_1, \pi_2)$, the second number that ranges from 1 to $N = n(n+1)/2$ represents another 185 two edges $e_i(\pi_i, \pi_{i+1}), e_j(\pi_j, \pi_{j+1})$, while this range can cover all possible 2 edges of 2-opt as shown



(a) ID of CUDA thread

(b) Index of 2opt



(c) Index of 4-opt

Figure 8: Indexes of GPU parallel 4-opt. (a) Simple numbers working as ID of CUDA threads; (b) Indexes of GPU parallel 2-opt where $N = n$; (c) Indexes of GPU parallel 4-opt where $N = n * (n + 1)/2$.

Algorithm 2 GPU parallel 4-opt and 5-opt.

Require: A TSP tour with n cities. We let $N = n(n + 1)/2$ and $\text{maxQuantity4optCheck} =$

$N(N + 1)/2$ and set CUDA kernel with fixed 1 dimensional BLOCKSIZE and GRIDSIZE.

```

1:  $tid \leftarrow \text{threadIdx.x} + \text{blockIdx.x} \times \text{blockDim.x}$ ;
2:  $\_\text{shared\_} \text{tspTour}[n]$ ;
3:  $\_\text{shared\_} \text{4optReconnectionSchemes}[25]$  or  $\text{5optReconnectionSchemes}[208]$ ;
4: Upload entire TSP tour, 25 4-opt or 208 5-opt re-connection schemes to shared memory;
5:  $id \leftarrow tid + \text{startBase}$ ; // one kernel launch takes responsible for a part of searching job.
6: if  $id < \text{maxQuantity4optCheck}$  then
7:    $X4, Y4 \leftarrow \text{function } \text{parse}(id)$ ;
8:    $o, p \leftarrow \text{function } \text{parse}(X4)$ ;
9:    $q, j \leftarrow \text{function } \text{parse}(Y4)$ ;
10:  if  $o < p < q < j$  then
11:    if node  $o|p|q|j$  has not already been occupied for any opt then
12:      if 4-opt then
13:        check all 25 4-opt re-connection schemes for edges  $e_o, e_p, e_q, e_j$ ;
14:        Mark selected 4-opt reconnection  $\text{schemeID}$ ; //first or best 4-opt
15:        break;
16:      end if
17:      if 5-opt then
18:        for each  $w > j$  &&  $w < n$  do
19:          if node  $w$  has not already been occupied for any opt then
20:            check all 208 5-opt reconnection schemes for edges  $e_o, e_p, e_q, e_j, e_w$ ;
21:            Mark selected 5-opt reconnection  $\text{schemeID}$ ; //first or best 5-opt
22:            break;
23:          end if
24:        end for
25:      end if
26:      if  $\text{schemeID} > 0$  then
27:        Store information of the selected 4-opt or 5-opt at city  $\pi_o$  on global memory;
28:        break; // Select the first 4-/5-opt.
29:      end if
30:    end if
31:  end if
32: end if

```

in Fig.8(a,b). So the first column in Fig.8(c) can cover all possible four edges starting from these two edges $e_0(\pi_0, \pi_1), e_1(\pi_1, \pi_2)$. The same situation happens to all other columns of Fig.8(c). So this method can cover all possible 4 edges of the global TSP tour. As to 5-opt $\mathcal{N}_5^{opqjw}$, we loop $j < w < n$ times of one set valid 4 edges with w acting as the fifth edge, so it also covers all possible 5 edges of the global TSP tour.

4.2.2. GPU parallel 6-opt

Given a TSP tour with n cities, there are totally $n(n-1)(n-2)(n-3)(n-4)(n-5)/720$ possible 6 edges that should be checked on the global tour, therefore we need to find a GPU parallel 6-opt solution that builds a connection between all these 6 edges and CUDA threads.

It is also challenging to solve a sextic equation with a coefficient of one dollar. Instead of solving this sextic equation, we propose a GPU parallel 6-opt algorithm that extends from Rocki's 3-opt and 2-opt algorithm.

Our proposed GPU parallel 6-opt method is shown in Fig.9. Supposing current TSP tour is oriented with n cities and n edges, all possible $n(n+1)(n+2)/6$ sets of 3 edges are distributed in Fig.9 (a), they can be mapped into CUDA threads easily same as Rocki's 3-opt method. For 6-opt, we let $N = n(n+1)(n+2)/6$ and map all $N(N+1)/2$ numbers into CUDA threads as shown in Fig.9(b), each box in Fig.9(b) represents two numbers $(X6, Y6)$, which finally represents six edges as shown in Fig.9(a) along current TSP tour. Once one CUDA thread gets one set of correct 6 edges, it computes all 1935 pure 6-opt re-connection schemes and accomplishes one 6-opt check. Its pseudocode is shown in Alg.3.

Here, to prove the proposed GPU parallel 6-opt $\mathcal{N}_6^{opqjw}$ method can cover all possible $n(n-1)(n-2)(n-3)(n-4)(n-5)/720$ six edges along the global TSP tour, we take an example in Fig.9, the two numbers in each box of the first column in Fig.9(b) represent six edges, the first number 0 represents three edges $e_0(\pi_0, \pi_1), e_1(\pi_0, \pi_1), e_2(\pi_1, \pi_2)$ as shown in Fig.9(a), the second number that ranges from 1 to $N = n(n+1)(n+2)/6$ represents other three edges $e_i(\pi_j, \pi_{j+1}), e_j(\pi_w, \pi_{w+1}), e_t(\pi_t, \pi_{t+1})$, while this range can cover all possible three edges same as Rocki's 3-opt method as shown in Fig.9(a). So the first column in Fig.9(b) can cover all possible 6 edges starting from these three edges $e_0(\pi_0, \pi_1), e_1(\pi_0, \pi_1), e_2(\pi_1, \pi_2)$. The same situation happens to all other columns of Fig.9(b). So the proposed methods can cover all possible 6 edges along the global TSP tour.

Algorithm 3 GPU parallel 6-opt $\mathcal{N}_6^{opqjw}$.

Require: A TSP tour with n cities. We let $N = n(n+1)(n+2)/6$ and $\text{maxQuantity6optCheck} = N(N+1)/2$, and set CUDA kernel with fixed 1 dimensional BLOCKSIZE and GRIDSIZE.

```

1:  $tid \leftarrow \text{threadIdx.x} + \text{blockIdx.x} \times \text{blockDim.x}$ ;
2:  $\_shared\_ \text{tspTour}[n]$ ;  $\_shared\_ \text{6optReconnectionSchemes}[1935]$ ;
3: Upload entire TSP tour and 1935 6-opt re-connection schemes to shared memory;
4:  $id \leftarrow tid + \text{startBase}$ ; // computed by max threads of one kernel launch and GRIDSIZE.
5: if  $id < \text{maxQuantity6optCheck}$  then
6:    $X6, Y6 \leftarrow \text{function } parse(id)$ ;
7:    $o, p, q \leftarrow \text{function } parse(X6)$ ;
8:    $j, w, t \leftarrow \text{function } parse(Y6)$ ;
9:   if  $o < p < q < j < w < t$  then
10:    Compute  $oldLength$  of edges  $(e_o, e_p, e_q, e_j, e_w, e_t)$ ;
11:    for each  $schemeID$  in 1935 6-opt re-connection schemes do
12:      Compute  $newLength$  of  $(e'_o, e'_p, e'_q, e'_j, e'_w, e'_t)$ ;
13:      if  $newLength < oldLength$  then
14:        Mark  $schemeID$ ; // first or best 6-opt
15:      end if
16:    end for
17:    if  $schemeID > 0$  then
18:      Store information of the selected 6-opt at city  $\pi_o$  on global memory;
19:      break; // Select the first 6-opt.
20:    end if
21:  end if
22: end if

```

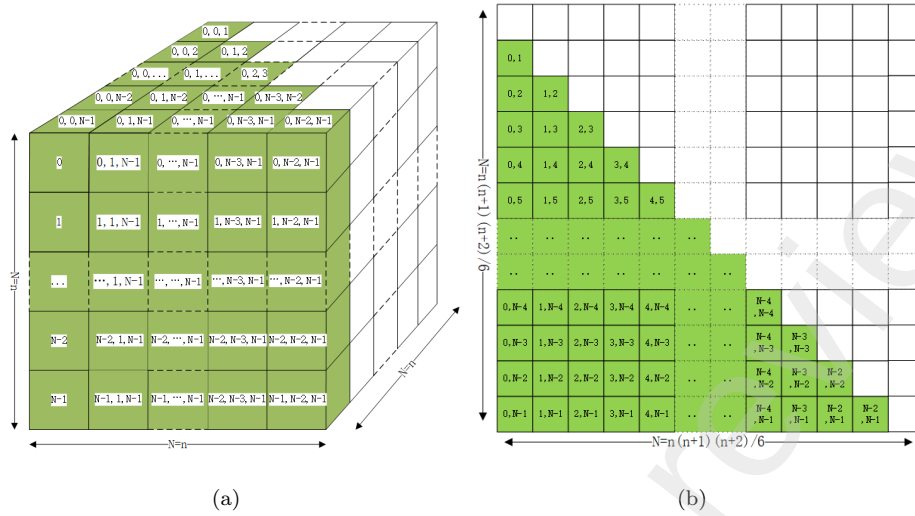


Figure 9: Indexes of GPU parallel 6-opt. (a) Indexes of GPU parallel 3-opt where $N = n$. (b) Indexes of GPU parallel 2-opt where $N = n(n+1)(n+2)/6$.

4.2.3. GPU parallel variable 2-/3-/4-/5-/6-opt

The above GPU parallel 4-opt 5-opt 6-opt algorithms together with Rocki's GPU parallel 2-opt 3-opt methods provide pure fixed k -opt ($2 \leq k \leq 6$) checks on the global TSP tour. However, they are not GPU parallel variable λ -opt ($2 \leq \lambda \leq 6$). In order to get variable λ -opt, it exists following two strategies:

- Iteratively apply mixed k -opt until achieving variable λ -optimal

Acting on the same initial TSP tour, try to search mixed k -opt ($2 \leq k \leq 6$) on the global TSP tour and execute non-interacted variable λ -opt ($2 \leq \lambda \leq 6$) on the same TSP tour. Iteratively repeating this procedure until achieving final variable λ -optimal status.

- Achieving increasing λ -optimal TSP tour starting with $\lambda=2$ and ending with $\lambda = 3, 4, 5, 6$

Starting from an initial TSP tour, firstly apply iterative 2-opt to achieve a 2-optimal TSP tour, then apply iterative increasing λ -opt ($2 < \lambda$) on previous $(\lambda-1)$ -optimal TSP tour until achieving λ -optimal TSP tour.

4.3. Judge non-interacted variable λ -opt

230 After one iteration of the above GPU parallel fixed or variable λ -opt ($2 \leq \lambda \leq 6$) searching on the global TSP tour, it may produce thousands of individual valid 2-/3-/4-/5-/6-opt moves that can improve current TSP tour independently. While, executing these 2-/3-/4-/5-/6-opt moves without selection may cut the current TSP tour into disjoint parts. In order to simultaneously execute multiple 2-/3-/4-/5-/6-opt moves on the same TSP tour, we need to judge non-interacted
235 2-/3-/4-/5-/6-opt moves with the least time complexity.

Here we propose a serial algorithm to select non-interacted mixed or fixed 2-/3-/4-/5-/6-opt moves on the same TSP tour. This algorithm takes linear time complexity and borrows ideas from our previous work in [17]. It has two cores, one is the criterion of **non-interacted variable 2-/3-/4-/5-/6-opt** moves, the other one is the algorithm to select these non-interacted 2-/3-/4-/5-/6-opt
240 moves according to that criterion.

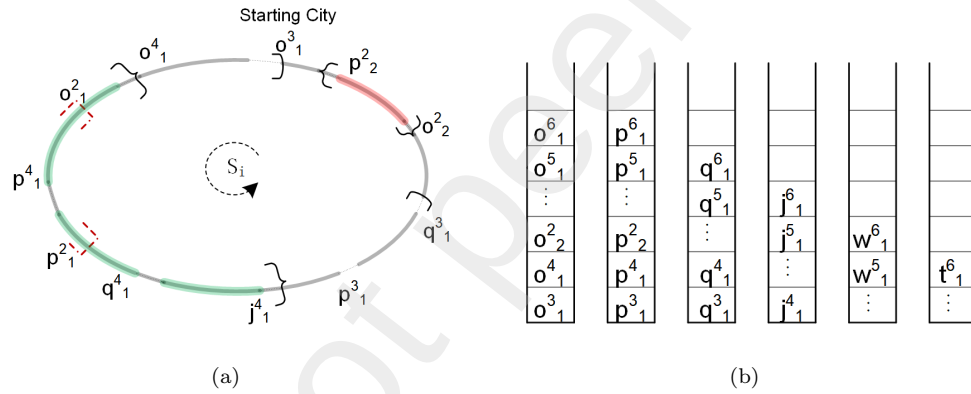


Figure 10: (a)The criterion to judge non-interacted mixed or fixed 2-/3-/4-/5-/6-opt moves on the same TSP tour s_i ; (b)We use six stacks to select non-interacted 2-/3-/4-/5-/6-opt moves on the same tour s_i .

4.3.1. Criterion of “non-interacted”

The criterion of “non-interacted variable 2-/3-/4-/5-/6-opt” moves takes advantage of current TSP tour order.

Supposing 2-opt moves $\mathcal{N}_2^{op}$, 3-opt moves $\mathcal{N}_3^{opq}$, 4-opt moves $\mathcal{N}_4^{opqj}$, 5-opt moves $\mathcal{N}_5^{opqjw}$ and
245 6-opt moves $\mathcal{N}_6^{opqjw t}$ are randomly distributed on the same TSP tour, where o, p, q, j, w, t represent TSP tour order and edges $e_o = (o, o + 1), e_p = (p, p + 1), e_q = (q, q + 1), e_j = (j, j + 1), e_w = (w, w + 1), e_t = (t, t + 1)$ represent these k edges that will be removed.

The sub-tour starting from city o to the last city $p|q|j|w|t$ of one $\mathcal{N}_2^{op}$ or $\mathcal{N}_3^{opq}$ or $\mathcal{N}_4^{opqj}$ or $\mathcal{N}_5^{opqjw}$ or $\mathcal{N}_6^{opqjw}$ respectively represents a tour range that should not be interrupted, which also means the criterion of “non-interacted” is that this sub-tour range should not cross these edges that will be removed. For example, as shown in Fig.10(a), the whole sub-tour range from city o_1^4 to j_1^4 of $\mathcal{N}_4^{o_1p_1q_1j_1}$ entirely locates in the sub-tour range from city o_1^3 to p_1^3 . In this case, if the 3-opt move $\mathcal{N}_3^{o_1p_1q_1}$ is selected firstly, then the 4-opt move $\mathcal{N}_4^{o_1p_1q_1j_1}$ can be selected as “non-interacted” move. On the contrary, if $\mathcal{N}_4^{o_1p_1q_1j_1}$ is selected firstly, then the $\mathcal{N}_2^{o_1p_1^2}$ can not be selected.

4.3.2. Algorithm to judge non-interacted variable 2-/3-/4-/5-/6-opt moves

Given the above criterion of “non-interacted”, we propose an algorithm to judge these non-interacted variable 2-/3-/4-/5-/6-opt moves while only traverse the tour s_i once, which means it takes linear time complexity. This algorithm uses 6 stacks as shown in Fig.10(b) and its algorithm flow is shown in Alg.4.

The core of this algorithm Alg.4 is that it traverses the TSP tour according to the same tour order used in the procedure of GPU parallel search 2-/3-/4-/5-/6-opt illustrated in section 4.2, and it only compares the newly met λ -opt move with the last selected variable λ -opt move whose information has been stored at top of these stacks. The function **JudgeNonInteracted** in Alg.4 judges whether a candidate k -opt move interacts with the previously selected one according to that criterion of “non-interacted” represented in section 4.3.1.

4.4. Deploy computation to multiple GPUs

Due to the reason that all these GPU parallel 2-/3-/4-/5-/6-opt checks are independent, our proposed GPU parallel λ -opt algorithm can be easily deployed on any quantity of GPU cards.

Given m GPU cards, it only needs to consider how to divide numbers from 0 to total quantity of 4-/5-/6-opt checks into m parts in order to balance each GPU’s work load. A general task division method is shown in Alg.5 that each GPU crossly executes same amount of these numbers. However, considering that a CUDA kernel can not launch infinity quantity of threads, we divide total quantity of 4-/5-/6-opt checks into parts and set the maximum k -opt checks one kernel launch can execute to be fixed as $maxQuantityKoptCheckDivided$ in Alg.5.

Side-effect of using multiple GPUs is that it needs to merge results from different GPU cards, and some k -opt searching computation is repeated thus not valuable due to one city can only

Algorithm 4 Sequentially select *non-interacted variable 2-/3-/4-/5-/6-opt moves* along current TSP solution s_i using linear time complexity.

Require: From s_i 's starting city π_0 , traverse each city π_o according to current tour order of s_i .

Consider a candidate $\mathcal{N}_2^{op}$ or $\mathcal{N}_3^{opq}$ or $\mathcal{N}_4^{opqj}$ or $\mathcal{N}_5^{opqjw}$ or $\mathcal{N}_6^{opqjw}$, $o < p < q < j < w < t$, $p \neq (o+1)$, $q \neq (p+1)$, $j \neq (q+1)$, $w \neq (j+1)$, $t \neq (w+1)$, stack A is for city π_o , stack B is for city π_p , stack C is for city π_q , similar with stacks D, E, F are for city π_j, π_w, π_t respectively.

```

1: for each city  $\pi_o$  from the starting city  $\pi_0$  do
2:   if  $A.size > 0$  &&  $\pi_o$  is in Stack B/C/D/E/F then // (City  $\pi_o$  has been occupied by
      previously selected k-opt move.)
3:     Mark A.top possessing a selected non-interacted k-opt move;
4:     From stack A to F, only pop k stacks' top elements.
5:   else if  $\pi_o$  possesses a  $\mathcal{N}_2^{op}$  or  $\mathcal{N}_3^{opq}$  or  $\mathcal{N}_4^{opqj}$  or  $\mathcal{N}_5^{opqjw}$  or  $\mathcal{N}_6^{opqjw}$  then
6:     Extract  $\pi_{p|q|j|w|t}$  from  $\pi_o$ 's information package;
7:     if Stacks A.size == 0 then
8:       Mark first selected k-opt move;
9:       Push  $\pi_{o|p|q|j|w|t}$  to stacks A|B|C|D|E|F respectively;
10:      Mark  $\pi_{p|q|j|w|t}$  is in stack B|C|D|E|F;
11:    else JudgeNonInteracted // (Compare the new k-opt with the last selected variable  $\lambda$ -opt
      move on top of these stacks.)
12:      if Non-interacted then
13:        Push  $\pi_{o|p|q|j|w|t}$  to stacks A|B|C|D|E|F respectively;
14:        Mark  $\pi_{p|q|j|w|t}$  is in stack B|C|D|E|F;
15:      end if
16:    end if
17:  else
18:    Continue next city  $\pi_{(o+1)}$  along  $s_i$ ; // (One edge only participate in one 2-opt move.)
19:  end if
20: end for

```

participate in one k-opt move and we do not use communication between GPU cards. But this side-effect can be ignored when quantity of GPU cards is really large.

Algorithm 5 A general task division method for parallel k-opt searching on m GPU cards.

Require: A TSP tour with n cities.

```

1: maxQuantityKoptCheck: represent the total numbers of all k-opt checks;
2: maxQuantityKoptCheckDivided: represent the max checks done by one kernel launch;
3: packSize = BLOCKSIZE * GRIDSIZE
4: checksOfOneThread  $\leftarrow$  maxQuantityKoptCheckDivided / packSize ;
5: maxStartBase  $\leftarrow$  maxQuantityKoptCheck / maxQuantityKoptCheckDivided;
6: if maxStartBase  $\leq$  1 then
7:   cudaSetDevice(0);
8:   kernel_parallel_Kopt(TSP, checksOfOneThread);
9: else
10:  for int startBase = 0; startBase < maxStartBase; startBase += m do
11:    cudaSetDevice(0)
12:    kernel_parallel_Kopt(TSP, checksOfOneThread, startBase);
13:    cudaSetDevice(1)
14:    kernel_parallel_Kopt(TSP, checksOfOneThread, startBase + 1);
15:    ...
16:    cudaSetDevice(m)
17:    kernel_parallel_Kopt(TSP, checksOfOneThread, startBase + m - 1);
18:  end for
19: end if

```

5. Experiments

Our goal is to achieve variable λ -optimal ($2 \leq \lambda \leq 6$) status of a given initial TSP tour quickly by using pure k-opt algorithm on the global TSP tour without considering nearest neighbor information in Euclidean space. This means the searching operation only has relation to the input quantity of “ n ” cities and does not have relation to Euclidean nearest neighbor information. This leads to two results: first one, the distribution of cities in Euclidean space does not influence performance of

285 pure k-opt algorithms; second one, it is not fair to compare with algorithms that consider Euclidean nearest neighbor information at the k-opt searching stage.

So, we test both serial and GPU parallel algorithms that purely search massive k-opt and execute multiple k-opt moves on the same global TSP tour. According to experiments, factors that impact the running time of GPU parallel 4-/5-/6-opt searching include:

- 290 • Limit of GPU on-chip shared memory, and whether code implementation uses GPU on-chip shared memory or not;
- Quality of the initial TSP tour, and rules to select the first or best k-opt optimization for one edge;
- Quantity of computation done by each CUDA thread, and data structures that influence
295 CUDA computation.

These factors also influence each other. For example, even if we assume quantity of GPU on-chip shared memory is large enough and put all intermediate variables on the on-chip shared memory, it does not necessarily run the fastest to achieve k-optimal under rules of “selecting the first k-opt optimization found for one edge”. For example, in Alg.2 if we adopt the “first” strategy that if
300 one city π_i has been occupied by the first candidate 4-/5-opt then all later 4/5 edges that include city π_i will not be checked again, and put the mark of which city has been occupied before on on-chip shared memory, this mark can only be visible for threads in current thread block, leading to the result of wasted computation by other thread blocks. However, if the old TSP tour is already optimized and each city has to try almost all 4/5 edges to search 4-/5-opt, putting that mark action
305 on on-chip shared memory will shorter overall running time.

Considering the above factors, we have implemented the following algorithms for comparison.

- **Serial-Multiple-Fixed-k-opt** implemented with a strategy that one iteration contains k loops of input n edges in order to find each edge’s first or best k-opt optimization. Then, multiple non-interacted fixed k-opt moves are selected and executed in one iteration. Their
310 algorithm flow is shown in Alg.6.
- **Serial-Multiple-Mixed-k-opt** implemented with a strategy that each edge tries to find its first (Alg.7) or best (Alg.8) k-optimization in current k’s loops, if it can not find k-opt, then it

tries $(k+1)$ -opt in the $(k+1)$'s loop. Then, multiple non-interacted mixed k-opt are selected and executed in one iteration.

- 315 • **GPU-Multiple-Fixed-k-opt** implemented with a strategy that each CUDA thread tries to find the edges' first or best k-opt optimization on GPU side with Alg.2 Alg.3. Then, non-interacted k-opt moves are selected and executed in one iteration.
- **GPU-Multiple-Mixed- λ -opt** implemented with a strategy each CUDA thread tries to find the edges' mixed 2-/3-/4-/5-/6-opt on GPU side. Then, multiple non-interacted mixed k-opt
320 are selected and executed on CPU side in one iteration.
- **Iterative-Fixed-k-opt**: Iteratively apply the above fixed k-opt algorithm until achieving fixed k-optimal status.
- **Iterative-Increase-k-optimal**: Achieving increasing λ -optimal TSP tour by starting with $\lambda=2$ and ending with $\lambda=3,4,5,6$.
- 325 • **Iterative-mixed- λ -opt**: Iteratively apply the above mixed k-opt algorithms until achieving variable λ -optimal status ($3 \leq \lambda \leq 6$).
- **Multi-GPUs-version**: Deploy the above methods on multiple GPU cards.

5.1. Experimental results

Experiments include performance comparison of the above mentioned different algorithms on
330 mu1979.tsp instance, and then we select the most efficient algorithm and test it on other 6 national TSP instances larger than 1979 provided by National TSP lib¹. All algorithms are implemented by C++ and on two computers, the first computer A has 12th Gen Intel(R) Core(TM) i7-12700H 2.3GHz and only runs serial algorithms, the second computer B has Intel(R) Xeon(R) CPU E5-2680 V4 with two NVIDIA RTX A4000 GPU and CUDA toolkit v12.6 with WDDM mode. CUDA kernel
335 configuration is set as $\langle\langle\langle 1024, 1048576 \rangle\rangle\rangle$. We use percentage deviation (%PD) compared with the optimum TSP tour provided by TSPLIB to mark the quality of an k-optimal TSP tour, and count the overall running time needed for an algorithm to achieve k-optimal TSP tour.

¹<https://www.math.uwaterloo.ca/tsp/world/countries.html>

Algorithm 6 Serial-Multiple-Fixed-k-opt: k is fixed to be 2 or 3 or 4 or 5 or 6.

Require: A TSP tour with n cities.

```
1: for  $o = 0; o < n; o++$  do
2:   for  $p = o + 1; p < n; p++$  do // 2-opt
3:     for  $q = p + 1; q < n; q++$  do // 3-opt
4:       for  $j = q + 1; j < n; j++$  do //4-opt
5:         for  $w = j + 1; w < n; w++$  do //5-opt
6:           for  $t = w + 1; t < n; t++$  do //6-opt
7:             oldLength = distance( $e(o, o + 1) + e(p, p + 1) + \dots$ );
8:             newLengths = distance(reconnection schemes of k-opt);
9:             if Select the “first” optimization then
10:               if oldLength > newLength then
11:                 Mark candidate k-opt for node  $o$ ;
12:                 break and begin next node  $o \leftarrow i + 1$ ;
13:               end if
14:             end if
15:             if Select the “best” optimization then
16:               optimization = oldLength - newLength;
17:               if new_optimization > old_optimization then
18:                 Mark candidate k-opt for node  $o$ ;
19:               end if
20:             end if
21:           end for
22:         end for
23:       end for
24:     end for
25:   end for
26: end for
27: for Node  $o$  with candidate  $k$ -opt;  $o++$  do
28:   Selecting non-interacted  $k$ -opt;
29: end for
```

Algorithm 7 Serial-Multiple-Mixed-k-opt with first strategy.

Require: A TSP tour with n cities.

```
1: for  $o = 0; o < n; o++$  do
2:   for  $p = o + 2; p < n; p++$  do
3:     Try 2-opt between edges  $e(o, o + 1), e(p, p + 1)$ ;
4:     if Exist 2-opt then
5:       break inner loops and begin next node  $o \leftarrow o + 1$ ;
6:     end if
7:     for  $q = p + 2; q < n; q++$  do
8:       Try 3-opt among edges  $e(o, o + 1), e(p, p + 1), e(q, q + 1)$ ;
9:       if Exist 3-opt then
10:        break inner loops and begin next node  $o \leftarrow o + 1$ ;
11:      end if
12:      for  $j = q + 2; j < n; j++$  do
13:        Try 4-opt among edges  $e(o, o + 1), e(p, p + 1), e(q, q + 1), e(j, j + 1)$ ;
14:        if Exist 4-opt then
15:          break inner loops and begin next node  $o \leftarrow o + 1$ ;
16:        end if
17:        for  $w = j + 2; w < n; w++$  do
18:          Try 5-opt among edges  $e(o, o + 1), e(p, p + 1), e(q, q + 1), e(j, j + 1), e(w, w + 1)$ ;
19:          if Exist 5-opt then
20:            break inner loops and begin next node  $o \leftarrow o + 1$ ;
21:          end if
22:          for  $t = w + 2; t < n; t++$  do
23:            Try 6-opt among edges  $e(o, o + 1), e(p, p + 1), \dots, e(w, w + 1), e(t, t + 1)$ ;
24:            if Exist 6-opt then
25:              break inner loops and begin next node  $o \leftarrow o + 1$ ;
26:            end if
27:          end for
28:        end for
29:      end for
30:    end for
31:  end for
32: end for
33: for Node  $i$  with candidate  $k$ -opt;  $i++$  do
34:   Select non-interacted variable 2-/3-/4-/5-/6-opt;
35: end for
```

Algorithm 8 Serial-Multiple-Mixed-k-opt with best strategy.

Require: A TSP tour with n cities.

```
1: for  $o = 0; o < n; o++$  do
2:   for  $p = o + 2; p < n; p++$  do
3:     Try 2-opt between edges  $e(o, o + 1), e(p, p + 1)$ ;
4:     if Exist 2-opt && (new_optimization > old_optimization) then
5:       Mark optimization;
6:     end if
7:     for  $q = p + 2; q < n; q++$  do
8:       Try 3-opt among edges  $e(o, o + 1), e(p, p + 1), e(q, q + 1)$ ;
9:       if Exist 3-opt && (new_optimization > old_optimization) then
10:        Mark optimization;
11:      end if
12:      for  $j = q + 2; j < n; j++$  do
13:        Try 4-opt among edges  $e(o, o + 1), e(p, p + 1), e(q, q + 1), e(j, j + 1)$ ;
14:        if Exist 4-opt && (new_optimization > old_optimization) then
15:          Mark optimization;
16:        end if
17:        for  $w = j + 2; w < n; w++$  do
18:          Try 5-opt among edges  $e(o, o + 1), e(p, p + 1), e(q, q + 1), e(j, j + 1), e(w, w + 1)$ ;
19:          if Exist 5-opt && (new_optimization > old_optimization) then
20:            Mark optimization;
21:          end if
22:          for  $t = w + 2; t < n; t++$  do
23:            Try 6-opt among edges  $e(o, o + 1), e(p, p + 1), \dots, e(w, w + 1), e(t, t + 1)$ ;
24:            if Exist 6-opt && (new_optimization > old_optimization) then
25:              Mark optimization;
26:            end if
27:          end for
28:        end for
29:      end for
30:    end for
31:  end for
32: end for
33: for Node  $i$  with candidate  $k$ -opt;  $i++$  do
34:   Select non-interacted variable 2-/3-/4-/5-/6-opt;
35: end for
```

Given the same initial TSP tour for mu1979.tsp, table1 shows the overall running time taken by different serial or parallel algorithms to achieve k-optimal TSP tour. In these tests, GPU parallel versions run on one GPU card and use on-chip shared memory for intermediat results. From table1, obvious contribution of our work lies in the following two aspects:

- Computation acceleration of GPU parallel 4-opt with “best” strategy;
- Computation acceleration of GPU parallel variable λ -opt ($3 \leq \lambda \leq 6$) with increasing λ -optimal strategy.

As to fixed 4-opt algorithms, From table1 , the iterative serial fixed 4-opt with “first” strategy (Alg.6) takes more than 56.742 hour to achieve a TSP tour with $pdb = 7.13$ that is close to 4-optimal status. It does not arrive exact 4-optimal status because the serial fixed 4-opt with “first” strategy (Alg.6) takes more than 48 hours to finish one iteration on that TSP tour with $pdb = 7.13$. This can be proved by the test of “iterative serial fixed 4-opt with best” strategy which takes more than 48 hours to run the first iteration. On the contrary, our proposed GPU parallel 4-opt with “first” strategy only takes 4.211 hour to achieve 4-optimal TSP tour, and our proposed GPU parallel 4-opt with “best” strategy only takes 2.405 hour to achieve 4-optimal TSP tour. This computation acceleration of more than 24 times (2.405h vs >56.742h) faster is achieved due the reason that one iteration of GPU parallel 4-opt only takes 53s (Fig.11 2nd column 3rd row), indicating that the GPU parallel 4-opt runs more than 2880 times faster (53s vs >48h) than serial version on an optimized TSP tour.

Detailed comparison between serial and parallel iterative fixed k-opt(k=2,3,4) is shown in Fig.11. These figures on left column of Fig.11 record convergence speed of four algorithms, namely the serial or parallel k-opt with first or best strategy and execute multiple k-opt moves on the same tour in one iteration. From these figures, we can see that the serial fixed k-opt algorithms with first strategy run faster than parallel versions at beginning iterations, while the parallel versions achieve the final k-optimal status more earlier. This is because one iteration of parallel k-opt actually considers all possible k edges, while serial fixed k-opt with first strategy only considers parts of all possible k edges at begining iterations. This can be proved by these figures on right column of Fig.11 which shows the time taken by a single k-opt searching step in each iteration, the serial k-opt search algorithm takes much more time than parallel version in later iterations of this procedure.

Table 1: The overall running time comparison to achieve k-optimal TSP tour for mu1979.tsp.				
PDB \Time	iterative serial fixed k-opt(first)	iterative serial fixed k-opt(best)	iterative parallel fixed k-opt(first)	iterative parallel fixed k-opt (best)
initial PDB	1143.63 \0	1143.63 \0	1143.63 \0	1143.63 \0
2-optimal	16.882 \2.011 s	15.487 \2.915 s	15.988 \2.2413s	16.082 \2.001 s
3-optimal	7.626 \5571.39 s	6.7573 \9610.945s	10.899 \1222.76s	9.19 \665.234 s
4-optimal	7.13 \ >56.742 h	1143.63 \>48h	7.055 \4.211 h	6.63 \2.405 h
5-optimal	20.396 \ > 60.721 h	1143.63 \>48h	1053.28 \>21.503 h	1013.98 \>32 h
6-optimal	81.749 \ > 37.053 h	1143.63 \>48h	1099.92 \>22.861 h	1100.68 \>13.84 h
PDB \Time	serial increasing k-optimal(first)	serial increasing k-optimal(best)	parallel increasing k-optimal(first)	parallel increasing k-optimal (best)
initial PDB	1143.63 \0	1143.63 \0	1143.63 \0	1143.63 \0
2-optimal	15.2726 \2.144 s	14.625 \3.216 s	16.406 \2.299 s	14.575 \1.933 s
3-optimal	7.413 \3464.42 s	9.243 \1370 s	10.078 \354.87 s	9.2684 \166.951 s
4-optimal	7.413 \>48 h	9.243 \>48 h	7.666 \1.605 h	<u>6.4158 \0.8312 h</u>
5-optimal	none \>72 h	none \>72 h	7.255 \>17.66 h	5.9253 \>37.531 h
6-optimal	none \>72 h	none \>72 h	6.957 \>21.35 h	5.6551 \>45.673 h
PDB \Time	serial variable mixed k-optimal(first)	serial variable mixed k-optimal(best)	parallel variable mixed k-optimal(first)	parallel variable mixed k-optimal(best)
initial PDB	1143.63 \0	1143.63 \0	1143.63 \0	1143.63 \0
2-optimal	none	none	none	none
3-optimal	9.356 \3398.98s	6.166 \24920.8s	10.525 \1087.635s	10.691 \654.190 s
4-optimal	87.869 \33.39 h	1143.63 \>48 h	7.895 \3.282 h	6.383 \2.077 h
5-optimal	809.393 \>48 h	1143.63 \>48 h	30.776 \>22.35 h	1146.63 \>24 h
6-optimal	953.539 \>63 h	1143.63 \>48 h	40.546 \>23.2 h	1143.63 \>24 h

As to fixed 5-/6-opt algorithm, since the quantity of all possible 5 or 6 edges is really large and one iteration of all possible 5 or 6 edges' 5-/6-opt searching checks all 5-/6-opt re-connection schemes, this procedure takes long time. So from table1, we can see that both serial and GPU parallel 5-/6-opt algorithms take long time. It is meaningless although the serial iterative fixed 5-/6-opt with first strategy runs relatively faster at the beginning iterations. The meaning of our proposed GPU parallel 5-opt and 6-opt algorithm lies in that, by using enough GPU cards, we can obtain 5-/6-opt optimization on the global TSP tour without considering nearest neighbor information.

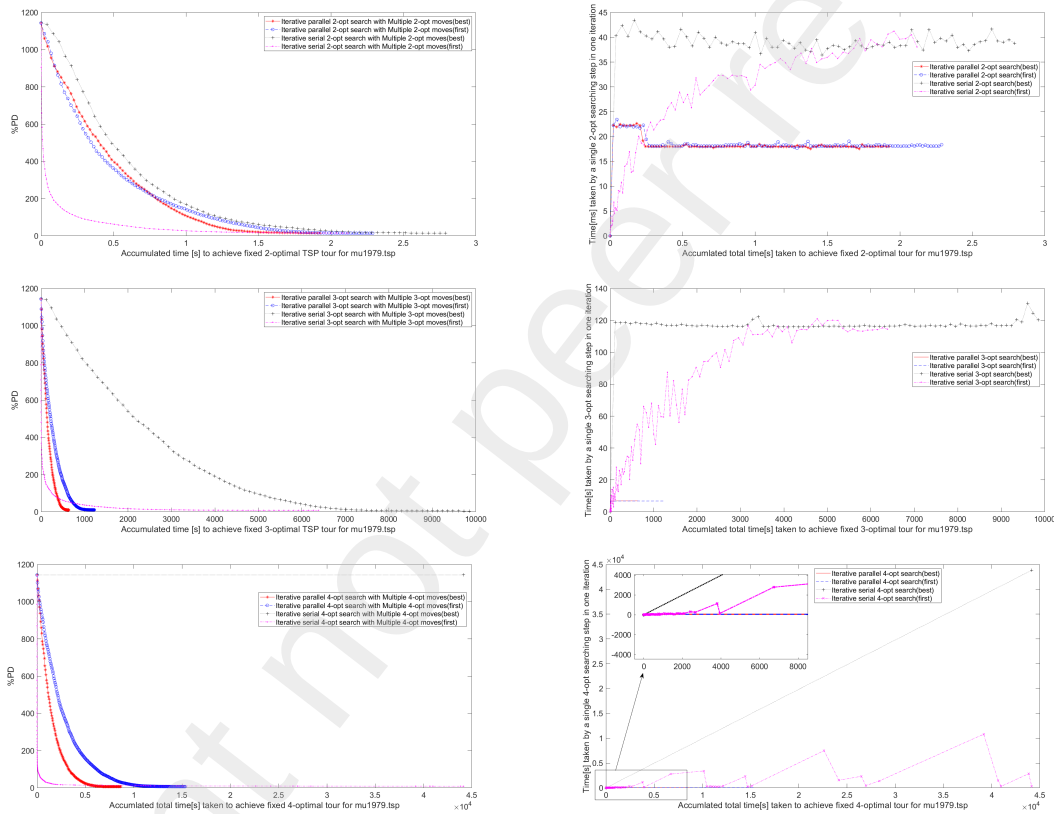


Figure 11: Time taken to achieve fixed k-optimal status by using iterative fixed k-opt algorithm for mu1979.tsp. Both serial and parallel implementations adopt strategies of “iteratively search fixed k-opt and execute multiple k-opt moves on the same TSP tour”, while serial 4-opt only achieves a status close to 4-optimal after a long time.

As to variable λ -opt algorithms, table1 records the total time taken by iterative increasing λ -

optimal starting with $\lambda=2$ and ending with $\lambda=3/4/5/6$. Our proposed GPU parallel increasing λ -optimal with best strategy only takes 0.8312 hour to achieve 4-optimal status for mu1979.tsp, and furthermore in acceptable time we can obtain some 5-/6-opt optimization on that 4-optimal tour. On the contrary, serial increasing λ -optimal algorithms both with first and best strategy can not obtain even one 4-opt optimization within 48 hour on a 3-optimal tour of mu1979.tsp. This GPU parallel increasing variable λ -opt algorithm runs more than 57 times (0.8312h vs >48h) faster than serial version.

Detailed comparison between serial and parallel iterative increasing λ -optimal methods is shown in Fig.12. The figure on left column of Fig.12 shows convergence speed of four algorithms to iteratively obtain increasing λ -optimal TSP tour, we can see that our proposed GPU parallel 2-/3-/4-opt with best strategy arrives 4-optimal TSP tour earliest. On the contrary, all serial algorithms can only arrive 3-optimal TSP tour and then no any more 4-opt optimization after long time. The figure on right column of Fig.12 shows the time taken by a single k-opt searching step in one iteration, our proposed GPU parallel 4-opt search on 1 GPU card takes less time than serial 3-opt algorithm.

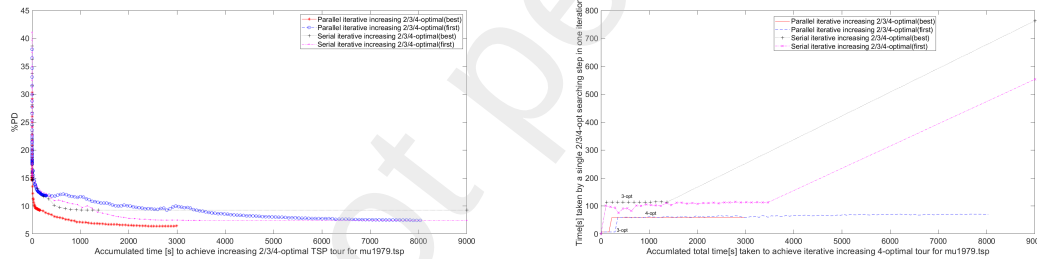


Figure 12: Time needed to achieve variable λ -optimal by increasingly obtain λ -optimal TSP tour for mu1979.tsp, which starts from $\lambda=2$ and ends with $\lambda=4$ by using parallel algorithm, while serial version only ends with $\lambda=3$ and can not achieve 4-optimal after a long time.

As to mixed k-opt algorithms, starting from the same initial TSP tour, we test iterative serial or parallel mixed 2-/3-opt 2-/3-/4-opt 2-/3-/4-/5-opt and 2-/3-/4-/5-/6-opt algorithms with the first or best strategy. It also shows that our proposed GPU parallel mixed k-opt algorithms run faster than serial versions, the parallel mixed 2-/3-/4-opt algorithm working on one GPU card takes 2.077h to achieve 2-/3-/4-optimal TSP tour while the serial version takes more than 33.39h to achieve a TSP tour with $pdb = 87.869$. The GPU parallel mixed λ -opt algorithm runs more than

30 times faster than serial version.

Detailed comparison between serial and parallel mixed 2-/3-opt and 2-/3-/4-opt is shown in Fig.13. Figures on the left column of Fig.13 also prove the fast convergence speed to achieve mixed 2-/3-optimal or 2-/3-/4-optimal TSP tour by using GPU parallel algorithms. Figures on right column of Fig.13 show that time taken at each iteration by parallel algorithms is far less than serial algorithms.

As to mixed 2-/3-/4-/5-opt and mixed 2-/3-/4-/5-/6-opt algorithm, also because the quantity of all possible 5 or 6 edges is really large and we consider all possible 5-/6-opt re-connection schemes, parallel algorithms using one or two GPU cards can not finish one iteration of all 5-/6-opt checks in acceptable time when working on an optimized TSP tour such as 4-optimal tour. These algorithms need more GPU cards to achieve that goal.

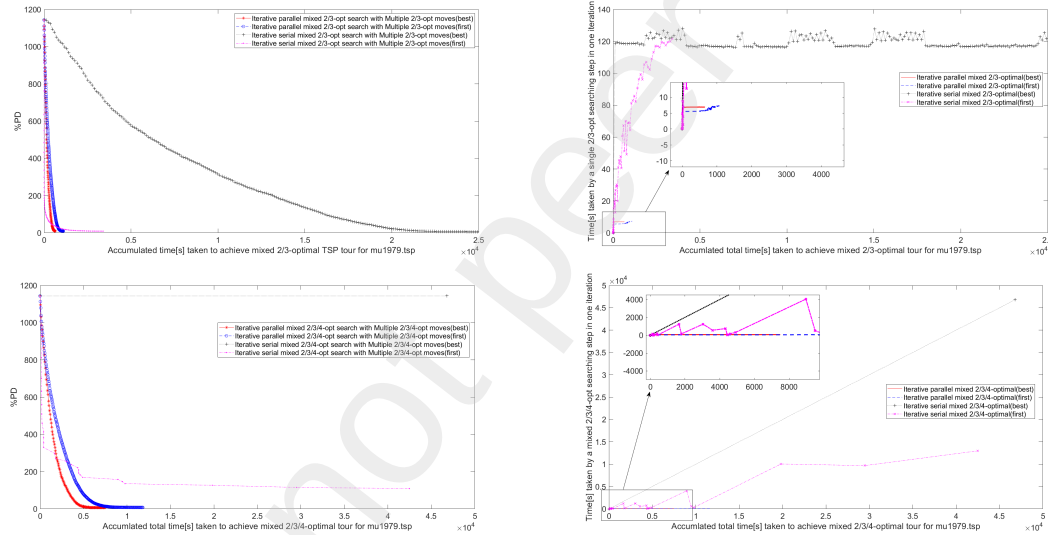


Figure 13: Time taken to achieve mixed 2-/3-optimal and 2-/3-/4-optimal status for mu1979.tsp. Both serial and parallel implementations adopt strategies of “iteratively search mixed k-opt and execute multiple mixed k-opt moves on the same TSP tour”, while serial version can not achieve mixed 2-/3-/4-optimal after a long time.

From the above experimental comparison, we get the fastest pure k-opt method to achieve 4-optimal TSP tour, namely the iteratively increasing λ -optimal using GPU parallel 2-/3-/4-opt search with best strategy and execute multiple k-opt on the same TSP tour. We test this method by using 1 or 2 GPU cards on TSP instance eg7146.tsp, its running time comparison with serial

version is shown in Fig.14. In this test, one iteration of serial 3-opt takes average 5394.49s on CPU, whereas one iteration of GPU parallel 3-opt takes average 26.28s; one iteration of serial 4-opt takes more than 48h, whereas one iteration of GPU parallel 4-opt takes average 790.545s on 1 GPU and 146.732s on 2 GPU cards without using explicit CUDA streams.

When using multiple GPUs, it is difficult to find a division method to exactly even these k-opt searching task done by each GPU according to our test. We tested the first division method shown in Alg.5 that crossly executes same amount of ID numbers by each GPU, and the second division method that divides the total ID numbers into t parts and one GPU takes $1/t$ parts of the task and leave the other GPU taking all remaining part. Experimental result shows that two GPU cards run simultaneously at beginning and there is always one GPU finishes its task earlier. We test 6 TSP instances by using both these two division methods and selecting the one that keeps two GPUs working simultaneously longer. Test result is shown in Fig.15, which proves that running time of one iteration parallel 3-/4-opt search can be further reduced by using more GPU cards.

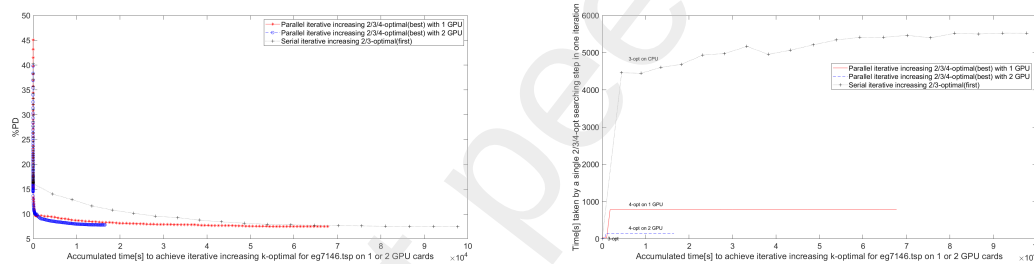


Figure 14: Time taken to achieve variable λ -optimal by using iterative increasingly λ -optimal algorithm for eg7146.tsp on 1 or 2 GPU cards, which starts from $\lambda=2$ and ends with $\lambda=4$ by using parallel algorithm, while serial version only ends with $\lambda=3$ and can not achieve 4-optimal after a long time.

6. Conclusion

In this paper, we focus on accelerating pure k-opt ($k=4,5,6$) and variable λ -opt ($\lambda = 2, 3, 4, 5, 6$) algorithms by using multiple GPU cards, these simple k-opt algorithms do not consider nearest neighbor information between any two cities and take long time when running serially.

Our proposed GPU parallel 4-opt 5-opt 6-opt algorithms search k-opt on the global TSP tour and can be easily deployed on any quantity of GPU cards. When only compare running time of one iteration of full 4-opt search based on an optimized TSP tour, the GPU parallel 4-opt runs

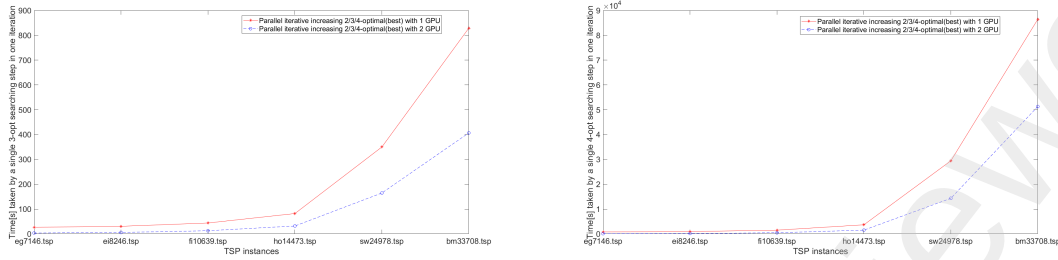


Figure 15: Average time taken by one iteration of GPU parallel 3-opt(left) 4-opt(right) on 1 or 2 GPU cards during the procedure of parallel iterative increasing λ -optimal for 6 TSP instances.

more than 2880 times faster than serial version on mu1979.tsp. And the proposed GPU 5-/6-opt algorithms can obtain some 5-/6-opt optimization in acceptable time which is impossible for serial version. When the goal is to achieve λ -optimal TSP tour by using pure variable λ -opt algorithm, we get the fastest pure k -opt method by using the proposed “multiple GPU parallel massive k -opt search and multiple variable k -opt ($k = 2, 3, 4, 5, 6$) moves on the same global TSP tour”.

Though our proposed GPU methods get computation acceleration versus their corresponding CPU versions, the overall running time taken to achieve k -optimal TSP tour is still relatively larger than other algorithms in literature that consider nearest neighbor information. Future work includes consideration of nearest neighbor information during k -opt searching procedure. Also with continuous improvement of GPU hardware, for example the increment of GPU on-chip shared memory, large TSP problems can be optimized to k -optimal status in much more shorter time.

7. Acknowledgements

This paper is sponsored by Beijing Municipal Education Commission with project ID: KM202111232002.

References

- [1] G. Laporte, The traveling salesman problem: An overview of exact and approximate algorithms, European Journal of Operational Research 59 (2) (1992) 231–247.
- [2] D. S. Johnson, L. A. McGeoch, The traveling salesman problem: A case study in local optimization, Local search in combinatorial optimization 1 (1997) 215–310.

- 450 [3] D. L. Applegate, R. E. Bixby, V. Chvatal, W. J. Cook, The traveling salesman problem: a computational study, Princeton university press, 2011.
- [4] M. R. Garey, D. S. Johnson, A guide to the theory of np-completeness, WH Freeman, New York 70 (1979).
- [5] K. Helsgaun, An effective implementation of the lin-kernighan traveling salesman heuristic, European Journal of Operational Research 126 (1) (2000) 106–130.
- 455 [6] W.-B. Qiao, J.-C. Créput, Component-based 2-/3-dimensional nearest neighbor search based on elias method to gpu parallel 2d/3d euclidean minimum spanning tree problem, Applied Soft Computing 100 (2021) 106928. doi:<https://doi.org/10.1016/j.asoc.2020.106928>. URL <https://www.sciencedirect.com/science/article/pii/S1568494620308668>
- 460 [7] G. Lancia, M. Dalpasso, Orbits, schemes and dynamic programming procedures for the tsp 4-opt neighborhood, arXiv preprint arXiv:2303.14424 (2023).
- [8] G. Lancia, M. Dalpasso, Finding the best 3-opt move in subcubic time, Algorithms 13 (11) (2020) 306.
- [9] R. M. Karp, Probabilistic analysis of partitioning algorithms for the traveling-salesman problem in the plane, Mathematics of operations research 2 (3) (1977) 209–224.
- 465 [10] M. Verhoeven, E. H. Aarts, P. Swinkels, A parallel 2-opt algorithm for the traveling salesman problem, Future Generation Computer Systems 11 (2) (1995) 175–182.
- [11] K. Rocki, R. Suda, Accelerating 2-opt and 3-opt local search using gpu in the travelling salesman problem, in: High Performance Computing and Simulation (HPCS), 2012 International Conference on, IEEE, 2012, pp. 489–495.
- 470 [12] K. Rocki, R. Suda, High performance gpu accelerated local optimization in tsp, in: Parallel and Distributed Processing Symposium Workshops & PhD Forum (IPDPSW), 2013 IEEE 27th International, IEEE, 2013, pp. 1788–1796.
- [13] W.-B. Qiao, J.-C. Créput, Parallel 2-opt local search on gpu, World Academy of Science, Engineering and Technology, International Journal of Electrical, Computer, Energetic, Electronic and Communication Engineering 11 (3) (2017) 281–285.
- 475

- [14] E. Rios, L. S. Ochi, C. Boeres, V. N. Coelho, I. M. Coelho, R. Farias, Exploring parallel multi-gpu local search strategies in a metaheuristic framework, *Journal of Parallel and Distributed Computing* 111 (2018) 39–55.
- 480 [15] W.-b. Qiao, J.-c. Créput, Massive parallel self-organizing map and 2-opt on gpu to large scale tsp, in: *International Work-Conference on Artificial Neural Networks*, Springer, 2017, pp. 471–482.
- [16] W.-B. Qiao, J.-C. Créput, Massive 2-opt and 3-opt moves with high performance gpu local search to large-scale traveling salesman problem, in: *International Conference on Learning and Intelligent Optimization*, Springer, 2018, pp. 82–97.
- 485 [17] W.-B. Qiao, J.-C. Créput, Multiple k- opt evaluation multiple k- opt moves with gpu high performance local search to large-scale traveling salesman problems, *Annals of Mathematics and Artificial Intelligence* 88 (4) (2020) 347–365.